

HADES

Database Security — Manuel complet (FR) / Complete manual (EN)
Le module db_scan explique a un debutant · The db_scan module explained for beginners

Pour tests de securite autorises uniquement / For authorised security testing only.

Scanner ou exploiter une base sans autorisation ecrite est illegal. Ce manuel explique, en francais et en anglais, ce que verifie le module Database Security et comment l'utiliser.

Scanning or exploiting a database without written permission is illegal. This manual explains, in French and English, what the Database Security module checks and how to use it.

Genere le / Generated 2026-06-04

Comment lire ce manuel / How to read this manual

Chaque section est presentee en DEUX colonnes : **le francais a gauche, l'anglais a droite**. Les lignes rose pale sont des sous-titres. Aucune connaissance prealable n'est requise.

Each section is shown in TWO columns: **French on the left, English on the right**. The pale-pink rows are sub-headings. No prior knowledge is required.

1 · Introduction — Qu'est-ce qu'une base de donnees ? / What is a database?

FRANCAIS	ENGLISH
Une base de donnees est le « coffre-fort » d'un site web : elle stocke les comptes, les mots de passe, les commandes, les messages — bref, toutes les donnees. Si quelqu'un y accede sans autorisation, c'est une fuite de donnees majeure, souvent irreversible.	A database is a website's 'vault': it stores accounts, passwords, orders, messages — all the data. If someone reaches it without authorisation, that is a major, often irreversible, data breach.
Les moteurs reconnus	The engines Hades knows
Hades reconnait MySQL / MariaDB, PostgreSQL, Microsoft SQL Server, Oracle, MongoDB, Redis, Elasticsearch, CouchDB, Cassandra et Memcached. Certains « parlent » le langage SQL (MySQL, PostgreSQL) ; d'autres sont dits « NoSQL » (MongoDB, Redis) et utilisent d'autres formats.	Hades knows MySQL / MariaDB, PostgreSQL, Microsoft SQL Server, Oracle, MongoDB, Redis, Elasticsearch, CouchDB, Cassandra and Memcached. Some 'speak' the SQL language (MySQL, PostgreSQL); others are called 'NoSQL' (MongoDB, Redis) and use different formats.
Aucune connaissance requise	No prior knowledge required
Ce manuel n'exige aucune base en cyber. Chaque terme technique est explique au moment ou il apparait, et un glossaire se trouve a la fin.	This manual assumes no security background. Every technical term is explained as it appears, and there is a glossary at the end.

2 · Pourquoi auditer la securite des bases de donnees / Why audit database security

FRANCAIS	ENGLISH
Une base exposee sur Internet est l'une des causes les plus frequentes de fuites massives : des millions de dossiers clients ont deja ete voles via des bases MongoDB ou Elasticsearch laisseees ouvertes, sans le moindre mot de passe. Des serveurs Redis exposes ont servi a installer des ranconiciels (ransomware).	A database exposed to the Internet is one of the most common causes of massive leaks: millions of customer records have been stolen from MongoDB or Elasticsearch databases left open with no password at all. Exposed Redis servers have been used to deploy ransomware.
Ce que cherche un attaquant	What an attacker looks for

FRANCAIS	ENGLISH
1) un port de base de donnees joignable depuis Internet ; 2) l'absence de mot de passe ; 3) des identifiants ecrits en clair dans un fichier ou le code du site ; 4) une injection SQL qui permet de lire la base via le site lui-meme. Le module db_scan teste exactement ces quatre points — et plus encore.	1) a database port reachable from the Internet; 2) the absence of a password; 3) credentials written in plain text in a file or the site's code; 4) a SQL injection that lets them read the database through the website itself. The db_scan module tests exactly these four things — and more.

3 · Le module db_scan de Hades / The Hades db_scan module

FRANCAIS	ENGLISH
db_scan est un profil de scan dedie : un seul module specialise (<code>scanner/db/db_security.py</code>) qui realise un audit complet de la surface « base de donnees » d'une cible, du debut a la fin, puis vous aide a exploiter ce qu'il trouve.	db_scan is a dedicated scan profile: a single specialised module (<code>scanner/db/db_security.py</code>) that performs a complete audit of a target's 'database' surface, end to end, then helps you exploit what it finds.
En une phrase	In one sentence
Il scanne les ports de bases de donnees, teste l'accès sans mot de passe (et extrait des donnees pour le prouver), cherche des injections SQL / NoSQL, traque les fichiers de secrets et identifiants fuites, puis calcule un score d'exposition, un plan d'attaque et un resume du « butin » recupere.	It scans database ports, tests password-free access (and extracts data to prove it), hunts for SQL / NoSQL injection, tracks down leaked secret and credential files, then computes an exposure score, an attack plan, and a summary of the 'loot' collected.
Detection d'abord, exploitation ensuite	Detection first, exploitation second
Par default, Hades ne fait que DETECTER. L'exploitation reelle (lancer le vrai outil sqlmap) n'a lieu que si vous ajoutez l'option <code>--exploit</code> et confirmez que vous etes autorise. Rien n'est jamais exploite automatiquement.	By default, Hades only DETECTS. Real exploitation (launching the actual sqlmap tool) happens only if you add the <code>--exploit</code> option and confirm you are authorised. Nothing is ever exploited automatically.

4 · Installation et prerequis / Installation and prerequisites

FRANCAIS	ENGLISH
Prerequis : Python 3.10 ou plus recent. Installez les dependances avec <code>pip install -r requirements.txt</code> . C'est tout pour la detection.	Requirements: Python 3.10 or newer. Install the dependencies with <code>pip install -r requirements.txt</code> . That is all you need for detection.
Optionnel : sqlmap	Optional: sqlmap
Pour l'exploitation des injections SQL (<code>--exploit</code>), installez sqlmap : <code>pip install sqlmap</code> . Hades le trouve automatiquement, meme s'il n'est pas dans le PATH (il regarde le dossier Scripts de Python).	For SQL-injection exploitation (<code>--exploit</code>), install sqlmap: <code>pip install sqlmap</code> . Hades finds it automatically, even if it is not on the PATH (it looks in Python's Scripts folder).

5 · Utilisation — les commandes / Usage — the commands

FRANCAIS	ENGLISH
Audit de base	Basic audit
<pre>py main.py --url https://cible.com --profile db_scan</pre> Lance l'audit base de donnees complet et affiche le resultat dans le terminal.	<pre>py main.py --url https://target.com --profile db_scan</pre> Runs the full database audit and prints the result in the terminal.
Avec un rapport HTML	With an HTML report
<pre>py main.py -u https://cible.com -p db_scan -o html</pre> Genere en plus un rapport HTML (theme sombre) avec le plan d'attaque et le butin.	<pre>py main.py -u https://target.com -p db_scan -o html</pre> Also generates an HTML report (dark theme) with the attack path and the loot.
Audit + exploitation des SQLi confirmees	Audit + exploiting confirmed SQLi
<pre>py main.py -u http://testaspnet.vulnweb.com -p db_scan --exploit</pre> Apres l'audit, propose de lancer sqlmap sur chaque injection SQL confirmee (double confirmation requise).	<pre>py main.py -u http://testaspnet.vulnweb.com -p db_scan --exploit</pre> After the audit, offers to launch sqlmap on each confirmed SQL injection (double confirmation required).
Astuce : testphp.vulnweb.com et testaspnet.vulnweb.com sont des sites de test publics (Acunetix) faits pour s'entrainer legalement.	Tip: testphp.vulnweb.com and testaspnet.vulnweb.com are public test sites (Acunetix) made for practising legally.

6 · Mode sur et legalite / Safe mode and legality

FRANCAIS	ENGLISH
N'auditez que des systemes que vous possédez ou que vous etes explicitement autorise a tester. Scanner ou exploiter une base sans autorisation ecrite est illegal dans la plupart des pays (CFAA, Computer Misuse Act, etc.).	Only audit systems you own or are explicitly authorised to test. Scanning or exploiting a database without written permission is illegal in most countries (CFAA, Computer Misuse Act, etc.).
Le mode sur (safe mode)	Safe mode
En mode sur, Hades saute les tests « destructifs » ou intrusifs : les identifiants par default, l'injection SQL temporisee (time-based) et l'injection NoSQL. Le scan de ports est limite aux 5 ports de bases de donnees les plus courants. Une note INFO vous le signale.	In safe mode, Hades skips the 'destructive' or intrusive tests: default credentials, time-based SQL injection, and NoSQL injection. The port scan is limited to the 5 most common database ports. An INFO note tells you so.

7 · Les verifications expliquees une par une / The checks explained one by one

FRANCAIS	ENGLISH
7.1 Scan de ports et empreinte	7.1 Port scan and fingerprint

FRANCAIS	ENGLISH
Hades essaie d'ouvrir une connexion vers les ports de bases de donnees connus (3306 MySQL, 5432 PostgreSQL, 6379 Redis, 27017 MongoDB, 9200 Elasticsearch...). Pour chaque port ouvert, il lit la « banniere » du service afin d'identifier le moteur et sa version. Si l'hote repond a TOUS les ports (pare-feu/leurre), le scan est ignore pour eviter de faux positifs.	Hades tries to open a connection to known database ports (3306 MySQL, 5432 PostgreSQL, 6379 Redis, 27017 MongoDB, 9200 Elasticsearch...). For each open port it reads the service 'banner' to identify the engine and its version. If the host answers EVERY port (firewall/honeypot), the scan is skipped to avoid false positives.
7.2 Acces sans authentication + extraction de donnees	7.2 Unauthenticated access + data extraction
Pour Redis, Memcached, Elasticsearch, CouchDB et MongoDB, Hades verifie s'il peut dialoguer SANS mot de passe. Si oui, il ne se contente pas de le dire : il extrait une preuve — un echantillon des cles Redis et leur nombre, les noms d'index Elasticsearch et le nombre de documents, la liste des bases CouchDB. C'est la preuve concrete qu'un attaquant pourrait tout lire. Severite : CRITIQUE.	For Redis, Memcached, Elasticsearch, CouchDB and MongoDB, Hades checks whether it can talk WITHOUT a password. If so, it does not just say it: it extracts proof — a sample of Redis keys and their count, Elasticsearch index names and document counts, the CouchDB database list. That is concrete proof an attacker could read everything. Severity: CRITICAL.
7.3 Redis CONFIG -> execution de code (RCE)	7.3 Redis CONFIG -> remote code execution (RCE)
Si un Redis ouvert autorise aussi la commande CONFIG, c'est bien pire qu'une fuite de donnees : un attaquant peut reecrire l'emplacement des fichiers de Redis pour déposer un « web shell », une cle SSH ou une tache planifiee, et ainsi prendre le controle total du serveur (RCE). Hades signale ce cas separement en CRITIQUE.	If an open Redis also allows the CONFIG command, it is far worse than a data leak: an attacker can rewrite where Redis stores its files to drop a 'web shell', an SSH key or a scheduled task, and thus fully take over the server (RCE). Hades flags this case separately as CRITICAL.
7.4 Injection SQL	7.4 SQL injection
Hades injecte des charges de test dans les parametres du site (URL et formulaires) pour voir si elles atteignent une requete de base de donnees. Trois techniques : par message d'erreur, par condition vraie/fausse (boolean aveugle) et par temporisation (une charge « dors 2 secondes » qui ralentit la page de facon mesurable). Severite : CRITIQUE. Une commande sqlmap prete a l'emploi est jointe.	Hades injects test payloads into the site's parameters (URL and forms) to see whether they reach a database query. Three techniques: via an error message, via a true/false condition (blind boolean), and via timing (a 'sleep 2 seconds' payload that slows the page measurably). Severity: CRITICAL. A ready-to-use sqlmap command is attached.
7.5 Injection NoSQL	7.5 NoSQL injection
Sur les bases NoSQL (type MongoDB), Hades envoie des operateurs speciaux (<code>{"\$ne": null}</code> , <code>{"\$gt": ""}</code> ...) et observe si la reponse change nettement (code ou taille). Un changement net suggere que l'entree est interpretee comme une requete — a verifier manuellement. Teste hors mode sur uniquement.	On NoSQL databases (e.g. MongoDB), Hades sends special operators (<code>{"\$ne": null}</code> , <code>{"\$gt": ""}</code> ...) and watches whether the response changes markedly (status or size). A clear change suggests the input is treated as a query — to be verified manually. Tested outside safe mode only.
7.6 Fichiers de secrets (.env, my.cnf...)	7.6 Secret files (.env, my.cnf...)

FRANCAIS	ENGLISH
Tres souvent, le mot de passe de la base se trouve dans un fichier de configuration laisse accessible : .env, config/database.yml, wp-config.php, my.cnf, appsettings.json, docker-compose.yml... Hades sonde une trentaine de ces chemins, lit le contenu et en extrait les identifiants de base de donnees. Le mot de passe est MASQUE dans le rapport. Severite : CRITIQUE.	Very often the database password sits in a configuration file left readable: .env, config/database.yml, wp-config.php, my.cnf, appsettings.json, docker-compose.yml... Hades probes about thirty such paths, reads the content, and extracts the database credentials. The password is MASKED in the report. Severity: CRITICAL.
7.7 Chaines de connexion fuites	7.7 Leaked connection strings
Hades lit aussi le code source des pages et des scripts du site a la recherche de chaines de connexion ecrites en dur (mongodb://user:pass@..., postgres://..., jdbc:...). Une seule de ces lignes peut suffire a se connecter directement a la base. Mot de passe masque, severite CRITIQUE.	Hades also reads the source of the site's pages and scripts looking for hard-coded connection strings (mongodb://user:pass@..., postgres://..., jdbc:...). A single such line can be enough to connect straight to the database. Password masked, severity CRITICAL.
7.8 Introspection GraphQL	7.8 GraphQL introspection
Si le site expose une API GraphQL avec « l'introspection » activee, n'importe qui peut telecharger le schema complet : toutes les requetes, mutations et types de donnees caches. Hades le detecte et liste les types exposes. Severite : ELEVEE.	If the site exposes a GraphQL API with 'introspection' enabled, anyone can download the full schema: every query, mutation and hidden data type. Hades detects it and lists the exposed types. Severity: HIGH.
7.9 Interfaces d'administration	7.9 Admin interfaces
Hades cherche les interfaces web de gestion de bases (phpMyAdmin, Adminer, pgAdmin, Fauxton de CouchDB, Kibana, mongo-express...). Une interface accessible en clair est CRITIQUE ; une interface presente mais protegee par mot de passe est ELEVEE (a retirer du web public si inutile).	Hades looks for database management web interfaces (phpMyAdmin, Adminer, pgAdmin, CouchDB's Fauxton, Kibana, mongo-express...). An interface reachable in the clear is CRITICAL; an interface that exists but is password-protected is HIGH (remove it from the public web if not needed).
7.10 Dumps et fichiers SQLite	7.10 Dumps and SQLite files
Hades tente de telecharger des sauvegardes de base laissees dans la racine web (backup.sql, dump.sql.gz, database.sqlite, .mdb...). Un fichier SQLite est reconnu par ses octets de signature. Un dump telechargeable contient souvent toute la base — donnees et mots de passe. Severite : CRITIQUE.	Hades tries to download database backups left in the web root (backup.sql, dump.sql.gz, database.sqlite, .mdb...). A SQLite file is recognised by its signature bytes. A downloadable dump often contains the whole database — data and passwords. Severity: CRITICAL.
7.11 Fuites par les frameworks	7.11 Framework leaks
Certains frameworks exposent des pages de debogage qui affichent la configuration, dont les identifiants de base (ex. Spring Actuator /actuator/env, Rails). Hades les sonde et signale toute fuite d'identifiants. Severite : CRITIQUE.	Some frameworks expose debug pages that print the configuration, including database credentials (e.g. Spring Actuator /actuator/env, Rails). Hades probes them and reports any credential leak. Severity: CRITICAL.
7.12 TLS sur les ports de base	7.12 TLS on database ports

FRANCAIS	ENGLISH
Hades verifie si la connexion a la base est chiffree (TLS). Un port ouvert sans TLS signifie que le trafic — identifiants compris — peut circuler en clair ; un certificat auto-signé est aussi signale. Severite : FAIBLE a MOYENNE.	Hades checks whether the database connection is encrypted (TLS). An open port without TLS means traffic — credentials included — may travel in clear text; a self-signed certificate is also flagged. Severity: LOW to MEDIUM.
7.13 Injection via en-tetes HTTP et cookies	7.13 Injection via HTTP headers and cookies
Les developpeurs oublient souvent que les EN-TETES (User-Agent, Referer, X-Forwarded-For) et les cookies finissent parfois dans une requete SQL (journalisation, statistiques). Hades injecte donc ses charges SQL aussi dans ces en-tetes et cookies, pas seulement dans les parametres d'URL — ce qui double la surface d'attaque. Severite : CRITIQUE si une erreur SQL en sort.	Developers often forget that HEADERS (User-Agent, Referer, X-Forwarded-For) and cookies sometimes end up in a SQL query (logging, analytics). So Hades injects its SQL payloads into these headers and cookies too, not only URL parameters — doubling the attack surface. Severity: CRITICAL if a SQL error comes out.
7.14 Bypass d'authentification NoSQL	7.14 NoSQL authentication bypass
Sur les applications MongoDB, un formulaire de connexion mal code peut etre trompe en envoyant un operateur au lieu d'un mot de passe : <code>`{"\$ne": ""}`</code> signifie « different de vide », donc « n'importe quel mot de passe ». Hades soumet ce type de charge aux formulaires de login et detecte si une session s'ouvre — c'est un contournement complet de l'authentification. Severite : CRITIQUE.	On MongoDB applications, a poorly coded login form can be tricked by sending an operator instead of a password: <code>`{"\$ne": ""}`</code> means 'not equal to empty', i.e. 'any password'. Hades submits this kind of payload to login forms and detects whether a session opens — a full authentication bypass. Severity: CRITICAL.
7.15 Bases de donnees cloud (Firebase, Firestore, Supabase)	7.15 Cloud databases (Firebase, Firestore, Supabase)
Beaucoup d'applis mobiles/web utilisent une base cloud dont l'adresse est ecrite dans le code. Hades la repere et teste si elle est lisible publiquement : une Firebase Realtime Database ouverte repond a l'adresse <code>`/.json`</code> avec TOUTES les donnees ; c'est l'une des fuites les plus frequentes du web moderne. Severite : CRITIQUE.	Many mobile/web apps use a cloud database whose address is written in the code. Hades spots it and tests whether it is publicly readable: an open Firebase Realtime Database returns ALL the data at the <code>`/.json`</code> address — one of the most common leaks of the modern web. Severity: CRITICAL.

8 · Comprendre le resultat / Understanding the output

FRANCAIS	ENGLISH
Le score d'exposition	The exposure score
A la fin, Hades calcule un « DB Exposure Score » de 0 a 100 et une note : SECURE (0-15), AT RISK (16-40), EXPOSED (41-70), CRITICAL (71-100). Plus le score est haut, plus la base est exposee. Chaque categorie de probleme ajoute des points une seule fois.	At the end, Hades computes a 'DB Exposure Score' from 0 to 100 and a grade: SECURE (0-15), AT RISK (16-40), EXPOSED (41-70), CRITICAL (71-100). The higher the score, the more exposed the database. Each category of problem adds points once.
Le panneau console	The console panel

FRANCAIS	ENGLISH
Un panneau dedie « Database Security Audit » affiche : la liste des trouvailles colorees par severite, la barre de score, un tableau recapitulatif (ports, problemes d'auth, points d'injection, fuites de donnees, interfaces), puis deux sections cles ci-dessous.	A dedicated 'Database Security Audit' panel shows: the list of findings coloured by severity, the score bar, a summary table (ports, auth issues, injection points, data leaks, interfaces), then the two key sections below.
Loot — le butin extrait	Loot — the extracted data
Cette section resume les DONNEES reellement recuperees pendant l'audit : echantillon de cles Redis, noms d'index Elasticsearch, bases CouchDB, types GraphQL, chaines de connexion et secrets fuites, identifiants par default. C'est la vision « ce qu'un attaquant repartirait avec ».	This section summarises the DATA actually retrieved during the audit: a sample of Redis keys, Elasticsearch index names, CouchDB databases, GraphQL types, leaked connection strings and secrets, default credentials. It is the 'what an attacker would walk away with' view.
Attack path — le plan d'attaque	Attack path — the exploitation plan
Hades transforme les failles exploitables en une liste ordonnee (de la plus grave a la moins grave) de commandes pretes a copier-coller : la ligne sqlmap pour chaque injection SQL, redis-cli pour Redis, curl pour Elasticsearch / CouchDB / fichiers de secrets. Un operateur peut reproduire l'acces etape par etape.	Hades turns exploitable findings into an ordered list (most to least severe) of ready-to-copy commands: the sqlmap line for each SQL injection, redis-cli for Redis, curl for Elasticsearch / CouchDB / secret files. An operator can reproduce the access step by step.
Le rapport HTML	The HTML report
Avec <code>-o html</code> , le rapport contient une section « Database Security » avec la jauge de score, le tableau des trouvailles, le plan d'attaque (commandes en vert) et le butin, plus une liste de remediation par moteur.	With <code>-o html</code> , the report contains a 'Database Security' section with the score gauge, the findings table, the attack path (commands in green) and the loot, plus a per-engine remediation list.

9 · Exploitation avec --exploit (sqlmap) / Exploitation with --exploit (sqlmap)

FRANCAIS	ENGLISH
Hades ne reinvente pas l'exploitation : il lance le vrai outil de reference, sqlmap, contre les injections SQL confirmees (issues de l'arsenal d'injection ET de db_scan).	Hades does not reinvent exploitation: it launches the real industry-standard tool, sqlmap, against confirmed SQL injections (from both the injection arsenal AND db_scan).
Comment ca se passe	How it works
Apres le scan, si une SQLi est confirmee, Hades affiche un panneau d'avertissement, puis demande DEUX confirmations : lancer sqlmap sur ce parametre, et confirmer que vous etes autorise. Ensuite seulement il execute sqlmap. Sans ces confirmations (ou sans <code>--exploit</code>), rien n'est exploite.	After the scan, if a SQLi is confirmed, Hades shows a warning panel, then asks for TWO confirmations: launch sqlmap on this parameter, and confirm you are authorised. Only then does it run sqlmap. Without those confirmations (or without <code>--exploit</code>), nothing is exploited.
Extraction active = preuve d'impact	Active extraction = proof of impact

FRANCAIS	ENGLISH
Avec <code>--exploit</code> , <code>db_scan</code> ne se contente plus de signaler : il EXTRAIT de vraies donnees pour prouver l'impact — valeurs reelles des cles Redis, documents Elasticsearch/CouchDB, et la banniere de la base via une injection SQL « in-band » (sans <code>sqlmap</code>). Tout est sauvegarde comme PREUVE dans le dossier <code>loot/<host>_<date>/</code> .	With <code>--exploit</code> , <code>db_scan</code> no longer just reports: it EXTRACTS real data to prove impact — actual Redis key values, Elasticsearch/CouchDB documents, and the database banner via an 'in-band' SQL injection (without <code>sqlmap</code>). Everything is saved as EVIDENCE in the <code>loot/<host>_<date>/</code> folder.
Reutilisation d'identifiants	Credential reuse
Si Hades a trouve des identifiants de base (dans un <code>.env</code> ou une chaine de connexion), il les REJOUe contre les serveurs de base de donnees decouverts pour verifier s'ils fonctionnent vraiment — confirmant un acces total. Le mot de passe reste masque dans le rapport.	If Hades found database credentials (in a <code>.env</code> or a connection string), it REPLAYS them against the discovered database servers to verify whether they actually work — confirming full access. The password stays masked in the report.
Rapport red-team (MITRE ATT&CK;)	Red-team report (MITRE ATT&CK;)
Chaque etape du plan d'attaque est etiquetee avec une technique MITRE ATT&CK; (ex. T1190, T1078) — le langage standard des equipes de securite — et pointe vers le fichier de preuve correspondant. Cela transforme le scan en un vrai compte-rendu d'engagement.	Each attack-path step is tagged with a MITRE ATT&CK; technique (e.g. T1190, T1078) — the standard language of security teams — and points to the matching evidence file. This turns the scan into a real engagement report.

10 · Glossaire / Glossary

FRANCAIS	ENGLISH
Port : porte numerotee d'un serveur par laquelle un service ecoute. Banniere : texte qu'un service renvoie en se presentant (nom, version). Authentification : verification d'identite (mot de passe, cle). Injection SQL : faire executer ses propres commandes a la base via un champ du site. NoSQL : bases sans langage SQL (MongoDB, Redis...). RCE : execution de code a distance = controle total du serveur. Dump : copie/sauvegarde complete d'une base. TLS : chiffrement des communications (le « s » de https).	Port: a numbered door on a server where a service listens. Banner: text a service returns when introducing itself (name, version). Authentication: identity verification (password, key). SQL injection: making the database run your own commands via a site field. NoSQL: databases without the SQL language (MongoDB, Redis...). RCE: remote code execution = full control of the server. Dump: a complete copy/backup of a database. TLS: encryption of communications (the 's' in https).

11 · Liste de controle de remediation / Remediation checklist

FRANCAIS	ENGLISH
• N'exposez jamais un port de base de donnees directement a Internet (VPN / liste blanche). • Exigez un mot de passe FORT et unique sur chaque base (jamais les valeurs par default). • Sortez les secrets de la racine web ; bloquez .env et fichiers de config au niveau du serveur. • Ne mettez jamais d'identifiants en dur dans le code front-end. • Retirez les interfaces d'admin (phpMyAdmin, Adminer...) de l'acces public. • Deplacez les dumps/sauvegardes hors de la racine web. • Utilisez des requetes parametrees contre l'injection SQL/NoSQL. • Desactivez l'introspection GraphQL en production. • Activez TLS pour les connexions a la base. • Apres toute fuite, changez immediatement le mot de passe concerne.	• Never expose a database port directly to the Internet (VPN / allowlist). • Require a STRONG, unique password on every database (never the default values). • Move secrets out of the web root; deny .env and config files at the server level. • Never hard-code credentials in front-end code. • Remove admin interfaces (phpMyAdmin, Adminer...) from public access. • Move dumps/backups out of the web root. • Use parameterised queries against SQL/NoSQL injection. • Disable GraphQL introspection in production. • Enable TLS for database connections. • After any leak, rotate the affected password immediately.